COMPTE-RENDU — TP 2 : Repérer les affirmations qui méritent une vérification (arabe)

Identité

Binôme ou individuel : INDIVIDUEL / SAKHI SARA

Filière / groupe : MBDIO /9 Date : 1/11/2026

Lien du dépôt : https://github.com/sakhisara/TP2-Reperer-les-affirmations-qui-meritent-une-verification-arabe

QUESTIONS DU COMPTE-RENDU

Q1. Distribution des labels dans train, val et test, globalement et par split (dev, dev-test, test) et présence d'un nombre. Scores du classifieur trivial « toujours No ». Quelle métrique retenez-vous, et pourquoi ?

Q1 — Distribution des labels et présence d'un nombre
dev = validation ; nombres détectés hors URL et mentions

	split	groupe	effectif	Yes	No	pourcentage_Yes
0	train	tous	7333	2243	5090	30.59
1	train	avec_nombre	2133	878	1255	41.16
2	train	sans_nombre	5200	1365	3835	26.25
3	dev	tous	1093	411	682	37.60
4	dev	avec_nombre	352	132	220	37.50
5	dev	sans_nombre	741	279	462	37.65
6	dev-test	tous	500	377	123	75.40
7	dev-test	avec_nombre	195	160	35	82.05
8	dev-test	sans_nombre	305	217	88	71.15
9	test	tous	610	218	392	35.74
10	test	avec_nombre	276	87	189	31.52
11	test	sans_nombre	334	131	203	39.22
12	Global	tous	9536	3249	6287	34.07
13	Global	avec_nombre	2956	1257	1699	42.52
14	Global	sans_nombre	6580	1992	4588	30.27

Ensemble	Effectif	Yes	No	% Yes
Train	7 333	2 243	5 090	30,59 %
Val = dev	1 093	411	682	37,60 %
Dev-test	500	377	123	75,40 %
Test	610	218	392	35,74 %
Total des quatre ensembles	9 536	3 249	6 287	34,07 %

	partie	modele	split	seed	seuil	precision_yes	rappel_yes	f1_yes	pr_auc	macro_f1	accuracy	temps_entrainement_s	notes
0	A	Toujours_No	dev	42	NaN	0.0	0.0	0.0	0.688	0.3842	0.624	0.0	Toutes les prédictions sont No.

Nous retenons principalement le **F1 de la classe Yes**, car l'objectif est de repérer les tweets méritant une vérification. L'accuracy seule peut être trompeuse : « toujours No » classe correctement 62,40 % des tweets de dev, mais ne détecte aucun Yes. Nous complétons donc le F1-Yes par la précision, le rappel et le macro-F1.

Q2. Raccourcis (Partie A) : quels indices superficiels sont corrélés aux labels ? Que valent vos classifieurs à une seule règle ? Qu'en concluez-vous sur la validité de vos scores ?

	split	indice	present	effectif	pourcentage_Yes
0	train	has_nombre	False	5200	26.25
1	train	has_nombre	True	2133	41.16
2	train	has_url	False	2488	24.28
3	train	has_url	True	4845	33.83
4	train	has_pays_ou_institution	False	4099	25.69
5	train	has_pays_ou_institution	True	3234	36.80
6	dev	has_nombre	False	741	37.65
7	dev	has_nombre	True	352	37.50
8	dev	has_url	False	394	39.34
9	dev	has_url	True	699	36.62
10	dev	has_pays_ou_institution	False	692	35.55
11	dev	has_pays_ou_institution	True	401	41.15

	partie	modele	split	seed	seuil	precision_yes	rappel_yes	f1_yes	pr_auc	macro_f1	accuracy	temps_entrainement_s	notes
0	A	Regle_nombre	dev	42	0.5	0.375	0.3212	0.346	0.4757	0.4977	0.5435	0.0	Yes si has_nombre=True ; score binaire.

	partie	modele	split	seed	seuil	precision_yes	rappel_yes	f1_yes	pr_auc	macro_f1	accuracy	temps_entrainement_s	notes
0	A	Regle_URL	dev	42	0.5	0.3662	0.6229	0.4613	0.5655	0.4527	0.4529	0.0	Yes si has_url=True ; score binaire.

	partie	modele	split	seed	seuil	precision_yes	rappel_yes	f1_yes	pr_auc	macro_f1	accuracy	temps_entrainement_s	notes
0	A	Regle_pays_institution	dev	42	0.5	0.4115	0.4015	0.4064	0.519	0.5278	0.559	0.0	Yes si has_pays_ou_institution=True ; score bi...

	partie	modele	split	seed	seuil	precision_yes	rappel_yes	f1_yes	pr_auc	macro_f1	accuracy	temps_entrainement_s	notes
0	A	TFIDF_LR_seuil_optimise	dev	42	0.1774	0.3921	0.9635	0.5574	0.4668	0.3676	0.4245	17.1947	Même modèle ; seuil maximisant F1-Yes sur dev.

Les règles simples obtiennent des scores non négligeables sans analyser réellement le sens des tweets. La règle URL atteint notamment un F1-Yes de 0,4613, contre 0,5574 pour TF-IDF + régression logistique au seuil optimisé. Une partie des performances peut donc provenir de raccourcis superficiels. Toutefois, leurs associations avec les labels varient entre train et dev, ce qui limite leur généralisation. Ces résultats ne rendent pas les scores invalides, mais imposent de les compléter par des ablations et des évaluations sur des données décalées.

Q3. Quels choix de prétraitement avez-vous faits par famille de modèles ? Lesquels détruisent des indices utiles(nombres, URL) ? Chiffres à l'appui (ablation).

J'ai adapté le prétraitement à chaque famille de modèles. Pour **TF-IDF + régression logistique**, j'ai appliqué une normalisation Unicode et arabe : suppression des diacritiques et du tatweel, harmonisation de certaines lettres et des espaces, et passage en minuscules. J'ai ensuite utilisé des n-grammes de mots 1–2 et de caractères 2–5 ; la branche caractères conserve notamment des traces de ponctuation, d'URL et de hashtags. Pour le **zero-shot NLI**, j'ai associé le texte original aux hypothèses, puis appliqué la tokenisation avec padding et troncature à 512 tokens. Pour la règle **« entité + nombre »**, j'ai détecté les entités avec CAMeLBERT et retiré les URL et mentions uniquement lors de la détection des nombres, afin de ne pas compter leurs chiffres. Pour **C1, C2 et C3**, j'ai normalisé l'arabe puis utilisé une tokenisation conservant les nombres, URL, mentions, hashtags et signes de ponctuation. J'ai construit le vocabulaire uniquement sur train, remplacé les tokens rares ou inconnus par <UNK>, puis appliqué le padding et la troncature. Pour **C4**, j'ai utilisé des séquences de caractères après normalisation arabe. Pour **AraBERT dans D et E1**, j'ai utilisé ArabertPreprocessor, puis le tokenizer du modèle avec une longueur maximale de 64 tokens. Enfin, pour **MARBERTv2 et XLM-R**, j'ai utilisé le texte original avec leur tokenizer respectif, le padding et la troncature ; XLM-R utilise une longueur maximale de 128 tokens.

F3 : réentraînement avec les url et nombres masqués :Les résultats suivants sont moyennés sur trois graines :

	variante	split	groupe	graines	effectif	Yes	No	f1_moyen	f1_ecart_type	precision_moyenne	rappel_moyen	pr_auc_moyen	macro_f1_moyen
0	nombres_masques	dev-test	avec_URL	3	306	243	63	0.8641	0.0214	0.9109	0.8244	0.9601	0.7203
1	nombres_masques	dev-test	avec_nombre	3	195	160	35	0.8395	0.0435	0.9090	0.7833	0.9469	0.6641
2	nombres_masques	dev-test	sans_URL	3	194	134	60	0.7761	0.0347	0.8060	0.7512	0.8752	0.6645
3	nombres_masques	dev-test	sans_nombre	3	305	217	88	0.8276	0.0118	0.8490	0.8095	0.9260	0.7160
4	nombres_masques	dev-test	tous	3	500	377	123	0.8326	0.0249	0.8730	0.7984	0.9330	0.6991
5	nombres_masques	test	avec_URL	3	345	138	207	0.5456	0.0106	0.4018	0.8502	0.4011	0.3968
6	nombres_masques	test	avec_nombre	3	276	87	189	0.5121	0.0123	0.3538	0.9272	0.3527	0.4316
7	nombres_masques	test	sans_URL	3	265	80	185	0.5234	0.0141	0.3644	0.9292	0.4018	0.4865
8	nombres_masques	test	sans_nombre	3	334	131	203	0.5563	0.0112	0.4141	0.8473	0.4460	0.4489
9	nombres_masques	test	tous	3	610	218	392	0.5368	0.0118	0.3864	0.8792	0.3952	0.4414
10	original	dev-test	avec_URL	3	306	243	63	0.8686	0.0213	0.8976	0.8450	0.9589	0.7111
11	original	dev-test	avec_nombre	3	195	160	35	0.8512	0.0405	0.9033	0.8083	0.9420	0.6690
12	original	dev-test	sans_URL	3	194	134	60	0.7940	0.0301	0.8050	0.7861	0.8727	0.6757
13	original	dev-test	sans_nombre	3	305	217	88	0.8353	0.0114	0.8379	0.8356	0.9281	0.7128
14	original	dev-test	tous	3	500	377	123	0.8419	0.0230	0.8639	0.8240	0.9317	0.6987
15	original	test	avec_URL	3	345	138	207	0.5507	0.0062	0.4041	0.8647	0.3977	0.3955
16	original	test	avec_nombre	3	276	87	189	0.5116	0.0094	0.3523	0.9349	0.3492	0.4231
17	original	test	sans_URL	3	265	80	185	0.5221	0.0117	0.3625	0.9333	0.3998	0.4808
18	original	test	sans_nombre	3	334	131	203	0.5613	0.0102	0.4167	0.8601	0.4459	0.4505
19	original	test	tous	3	610	218	392	0.5393	0.0081	0.3870	0.8899	0.3943	0.4384

F4 : suppressions des nombres et des entités des tweets Yes bien détectés et la mesure de la chute.

	split	variante	graines	chute_moyenne_points	chute_ecart_type	baisse_score_moyenne
0	dev-test	sans_entites	3	2.4055	0.8312	-0.0027
1	dev-test	sans_nombres	3	0.2099	0.1830	-0.0020
2	dev-test	sans_nombres_et_entites	3	2.4897	0.5158	-0.0057
3	test	sans_entites	3	9.1900	3.9461	0.0914
4	test	sans_nombres	3	0.1783	0.3087	0.0025
5	test	sans_nombres_et_entites	3	9.7059	3.9627	0.0971

Q4. Zero-shot (B1) : score obtenu et sensibilité à la formulation des hypothèses. Pourquoi un modèle NLI peut-il classer sans exemple d'entraînement ? Quel rôle joue l'attention entre prémisse et hypothèse ?

J'ai utilisé **mDeBERTa-v3-base-XNLI** avec trois formulations des hypothèses, en arabe, français et anglais.

	modele	seuil	precision_yes	rappel_yes	f1_yes	pr_auc	macro_f1
0	NLI_ar_050	0.5000	1.0000	0.0024	0.0049	0.3656	0.3869
1	NLI_ar_optimise	0.0013	0.3760	1.0000	0.5465	0.3656	0.2733
2	NLI_fr_050	0.5000	0.3571	0.6837	0.4691	0.3488	0.4127
3	NLI_fr_optimise	0.0142	0.3764	1.0000	0.5469	0.3488	0.2749
4	NLI_en_050	0.5000	0.7500	0.0073	0.0145	0.3874	0.3918
5	NLI_en_optimise	0.0005	0.3760	1.0000	0.5465	0.3874	0.2733

Au seuil standard de **0,5**, les résultats montrent une forte sensibilité à la langue et à la formulation des hypothèses. Les hypothèses françaises donnent un **F1-Yes de 0,4691**, contre **0,0049 en arabe** et **0,0145 en anglais**. Avec les hypothèses arabes et anglaises, le rappel est presque nul : le modèle prédit très rarement Yes. La précision arabe de 1,0000 doit donc être relativisée, puisqu'elle concerne un nombre infime de prédictions positives.

Après optimisation du seuil sur dev, les F1 deviennent proches : **0,5465 en arabe et en anglais**, et **0,5469 en français**. Les seuils très faibles conduisent cependant le modèle à prédire presque tous les tweets comme Yes. Le rappel atteint ainsi **1,0000**, mais la précision reste autour de **0,376**, soit la proportion de Yes dans dev. **L'amélioration du F1 vient donc surtout de l'augmentation du rappel, et ne démontre pas une bonne distinction entre les classes.** Le faible macro-F1 après optimisation confirme ce déséquilibre des performances entre Yes et No.

Q5. La règle « entité + nombre ⇒ Yes » (B2) : précision, rappel, F1. Que révèle-t-elle de la tâche ?

J'ai appliqué la règle **« prédire Yes si le tweet contient au moins une entité nommée et un nombre>>**

Détection des entités: 100% 69/69 [00:15<00:00, 5.84it/s]

[transformers] You seem to be using the pipelines sequentially on GPU. In order to maximize efficiency please use a dataset

	partie	modele	split	seed	seuil	precision_yes	rappel_yes	f1_yes	pr_auc	average_precision	macro_f1	accuracy	temps_entrainement_s	temps_inference_s	pourcentage_predit_Yes	notes
0	B2	CAMeLBERT_entite_et_nombre	dev	42	0.5	0.3769	0.3017	0.3351	0.4706	0.3763	0.4974	0.5499	0.0	15.426	30.1006	Yes si au moins une entité NER et un nombre ho...

Seulement 37,69 % des tweets prédits Yes sont réellement Yes, et la règle ne détecte que 30,17 % des Yes réels. Ces résultats montrent que la présence conjointe d'une entité et d'un nombre ne suffit pas à déterminer si un tweet mérite une vérification. Un tweet peut contenir ces éléments sans présenter une affirmation importante à vérifier ; inversement, une affirmation vérifiable peut ne contenir aucun nombre. La tâche nécessite donc une analyse du sens et du contexte.

Q6. Déséquilibre : comparez pondération de la perte, sur-échantillonnage et optimisation du seuil (tableau). Pourquoi le sur-échantillonnage doit-il être fait après la séparation train / val ?

Q6 — C3 sans positions, AraVec gelé — Validation dev

Stratégie	F1-Yes à 0,5	Seuil optimisé	F1-Yes optimisé
Perte standard	0.1566	0.0165	0.5693
Perte pondérée	0.4943	0.0154	0.5694
Sur-échantillonnage	0.3924	0.0654	0.5685

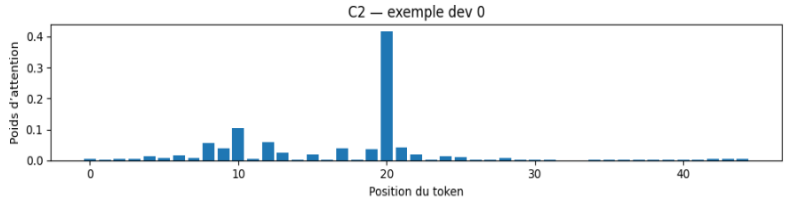
j'ai comparé trois stratégies sur la même architecture : **C3** sans encodage positionnel, avec **AraVec gelé.** La pondération de la perte pénalise davantage les erreurs sur la classe Yes, minoritaire, tandis que le sur-échantillonnage présente plus fréquemment ses exemples pendant l'entraînement. Au seuil de 0,5, la perte pondérée obtient le meilleur **F1-Yes (0,4943)**, devant **le sur-échantillonnage (0,3924)** et la perte standard **(0,1566).**Après optimisation du seuil sur dev, les trois stratégies donnent des résultats presque identiques : **0,5693, 0,5694 et 0,5685**. Le choix du seuil joue donc un rôle majeur dans cette expérience. La pondération et le sur-échantillonnage n'apportent presque aucun gain supplémentaire sur le F1 optimisé.Le sur-échantillonnage doit être effectué après la séparation train/val, uniquement sur train. Sinon, des copies d'un même exemple pourraient apparaître dans les deux ensembles, créant une fuite de données et des scores de validation trop optimistes. La validation doit conserver sa distribution d'origine pour permettre une évaluation représentative.

Q7. C1 vs C2 vs C3 (et C4) : résultats. Les poids d'attention se concentrent-ils sur les nombres, les entités nommées et les verbes de déclaration ? Effet de l'encodage positionnel ?

Modèle visualisé : C2_aravec_fige

Tweet : د. سعد الفقيه @saadalfagih تفاصيل كثيرة ومهمة حول الموضوع "المتداول في الدوائر المقربة من #السعودية و #قطر أن الأخيرة تصر بحدم المشاركة في #القمة_الخليجية بأميرها إن لم تفتح الحدود والأجواء ويرفع الحصار وستكون المشاركة بتمثيل أدنى" على قناة الحوار: https://t.co/mZNSLBv9jM
Label réel : Yes

	position	token_original	token_modele	attention
20	20	تصر	تصر	0.417893
10	10	المتداول	المتداول	0.104283
12	12	الدوائر	الدوائر	0.057997
8	8	:	:	0.056445
21	21	بحدم	بحدم	0.043142
9	9	"	"	0.038912
17	17	قطر#	قطر#	0.038040
19	19	الاخيره	الاخيره	0.036322
13	13	المقربة	<UNK>	0.025098
22	22	المشاركة	المشاركة	0.020327



J'ai comparé C1, C2, C3 et C4 sur dev, avec un seuil optimisé pour chaque modèle. Les meilleures configurations obtiennent un F1-Yes de 0,5682 pour C1 avec AraVec dégelé, 0,5692 pour C2 avec AraVec gelé, 0,5693 pour C3 sans positions avec AraVec gelé, et 0,5569 pour C4 Char-CNN. C1, C2 et C3 présentent donc des performances très proches, tandis que C4 est légèrement en retrait. À embeddings identiques, l'ajout de l'attention améliore le BiLSTM avec AraVec gelé : le F1 passe de 0,5627 pour C1 à 0,5692 pour C2.

L'observation des poids d'attention de C2 montre que, dans cet exemple, le poids le plus élevé concerne le verbe « تصر » (« insiste »), avec 0,4179, suivi de « المتداول » (« ce qui circule »), avec 0,1043. L'entité « قطر# » (« Qatar ») reçoit un poids de 0,0380, tandis que les deux-points obtiennent 0,0564. L'attention privilégie donc ici une expression liée à l'affirmation rapportée, tout en accordant du poids à d'autres mots et à la ponctuation. Cet exemple ne permet pas de conclure à une concentration systématique sur les entités, ni d'évaluer l'attention portée aux nombres. Ces observations restent qualitatives et ne constituent pas une preuve de l'importance causale des tokens.

Enfin, avec AraVec gelé, C3 obtient 0,5664 avec encodage positionnel, contre 0,5693 sans positions, soit une amélioration de 0,29 point de F1 après suppression des positions. L'encodage positionnel n'apporte donc pas de gain dans cette expérience. Toutefois, cet écart est faible et mesuré sur une seule graine : il ne permet pas de conclure que l'ordre des mots est généralement inutile.

Q8. PLM gelé (D) vs fine-tuné (E1) : gain et coût (temps, paramètres entraînés). aubmindlab/bert-base-arabertv02-twitter vs UBC-NLP/MARBERTv2 : lequel, et pourquoi ?

	partie	modele	lambda_en	graines	f1_moyen	f1_ecart_type	pr_auc_moyen	macro_f1_moyen	temps_moyen_s
0	E1	E1_AraBERT_optimise	0.0	3	0.6038	0.0096	0.5852	0.6266	293.9673
1	E1	E1_MARBERTv2_optimise	0.0	3	0.6010	0.0059	0.5732	0.6603	323.6512

	partie	modele	split	seed	temps_entrainement_s	seuil	precision_yes	rappel_yes	f1_yes	macro_f1	pr_auc
0	D	D1_CLS_050	dev	42	11.546	0.5000	0.5068	0.4526	0.4781	0.5949	0.4863
1	D	D1_CLS_optimise	dev	42	11.546	0.0689	0.4520	0.8370	0.5870	0.5549	0.4863

	partie	modele	split	seed	temps_entrainement_s	seuil	precision_yes	rappel_yes	f1_yes	macro_f1	pr_auc
0	D	D1_moyenne_050	dev	42	9.1478	0.5000	0.5239	0.5328	0.5283	0.6201	0.5223
1	D	D1_moyenne_optimise	dev	42	9.1478	0.0943	0.4346	0.8735	0.5804	0.5168	0.5223

Nombre de paramètres entraînables :

modele	parametres_entrainables
E1_AraBERT_050	135,194,113
E1_AraBERT_optimise	135,194,113
E1_MARBERTv2_050	162,842,113
E1_MARBERTv2_optimise	162,842,113
F3_E1_AraBERT_nombres_masques_050	135,194,113
F3_E1_AraBERT_nombres_masques_optimise	135,194,113

J'ai comparé un encodeur préentraîné gelé à un encodeur fine-tuné. Dans D, les paramètres d'AraBERT restent fixes : seuls le classifieur ou la tête BiLSTM-attention sont entraînés. Dans E1, les paramètres de l'encodeur et de la tête de classification sont ajustés. Le fine-tuning d'AraBERT atteint un F1-Yes moyen de 0,6038, contre 0,5922 pour D2, soit un gain de 1,16 point de F1, pour un coût d'entraînement supérieur. Les temps de D excluent toutefois l'extraction préalable des représentations et ne constituent donc pas son coût total.

Entre aubmindlab/bert-base-arabertv02-twitter et UBC-NLP/MARBERTv2, je retiens AraBERT, car il obtient un F1-Yes moyen légèrement supérieur sur dev (0,6038 contre 0,6010) avec un temps d'entraînement plus faible (293,97 s contre 323,65 s). Cet avantage reste modeste et ne suffit pas à affirmer une supériorité générale.

Q9. Multi-tâche (E2) : l'ajout des données anglaises améliore-t-il la tâche principale ? Effet de λ.

Pourquoi une tâche auxiliaire peut-elle aider… ou nuire ?

Dans E2, j'ai utilisé **XLM-R avec un encodeur partagé et deux têtes de classification**, arabe et anglaise. Le coefficient **λ contrôle la contribution de la perte anglaise L = L_arabe + λ × L_anglais.**

2	E2	E2_XLMR_lambda_0.5_optimise	0.5	3	0.5957	0.0081	0.5425	0.6332	805.0873
4	E2	E2_XLMR_lambda_1_optimise	1.0	3	0.5864	0.0043	0.5371	0.6233	790.2782
3	E2	E2_XLMR_lambda_0_optimise	0.0	3	0.5844	0.0092	0.5339	0.5825	539.9175

```
Lambda retenu sur dev arabe : 0.5
```

Dans E2, j'ai étudié l'effet des données anglaises sur la classification des tweets arabes en faisant varier λ, qui pondère la perte anglaise : L = L_arabe + λ × L_anglais. Sans contribution anglaise (λ = 0), le F1-Yes moyen sur dev arabe est de 0,5844. Avec λ = 0,5, il atteint 0,5957, soit un gain de 1,13 point de F1. Avec λ = 1, il redescend à 0,5864. L'anglais apporte donc une amélioration modeste, maximale parmi les valeurs testées pour λ = 0,5, que j'ai retenu sur dev arabe. La tâche auxiliaire peut aider en enrichissant les représentations de l'encodeur partagé grâce à des caractéristiques communes aux affirmations arabes et anglaises, tout en limitant le surapprentissage. Elle peut cependant nuire lorsque les données diffèrent par leurs thèmes ou leurs annotations, ou lorsque les mises à jour favorisent l'anglais au détriment de l'arabe. Le recul observé entre λ = 0,5 et λ = 1 montre ici qu'augmenter la contribution anglaise n'améliore pas systématiquement la tâche principale.

Q10. Décalage de prior (F1) : part de Yes dans dev, dev-test et test. Écart entre le seuil choisi sur dev et le seuil optimal a posteriori sur dev-test et test. Que mettriez-vous en place en production ?

	split	effectif	pourcentage_Yes
0	dev	1093	37.60
1	dev-test	500	75.40
2	test	610	35.74

	modele	split	seed	seuil_dev	seuil_optimal_posteriori	ecart_seuil	f1_seuil_dev	f1_optimal_posteriori
0	E1_AraBERT	dev-test	42	0.0336	0.0035	-0.0301	0.8685	0.8842
1	E1_AraBERT	test	42	0.0336	0.2496	0.2160	0.5393	0.5465
2	E1_AraBERT	dev-test	43	0.2445	0.0196	-0.2249	0.8293	0.8951
3	E1_AraBERT	test	43	0.2445	0.7268	0.4823	0.5475	0.5570
4	E1_AraBERT	dev-test	44	0.5360	0.1930	-0.3430	0.8278	0.8902
5	E1_AraBERT	test	44	0.5360	0.0292	-0.5068	0.5312	0.5418

J'ai retenu E1 AraBERT, car il obtient le meilleur F1-Yes moyen sur dev parmi les configurations E : 0,6038, contre 0,6010 pour MARBERTv2 et 0,5957 pour E2. Ce choix repose uniquement sur la validation.La proportion de Yes est de 37,60 % sur dev, 75,40 % sur dev-test et 35,74 % sur test. Dev-test présente donc un fort décalage de prior. Avec les seuils choisis sur dev, les F1 varient de 0,8278 à 0,8685 sur dev-test, contre 0,5312 à 0,5475 sur test. La forte proportion de Yes sur dev-test contribue au F1 élevé, sans prouver à elle seule une meilleure généralisation.

Les seuils optimaux a posteriori sont plus faibles sur dev-test et variables sur test. Sur test, leur gain reste limité à 0,72–1,06 point de F1 : le seuil seul n'explique donc pas l'écart entre les ensembles. Ces seuils servent uniquement à l'analyse ; pour l'évaluation, je conserve ceux choisis sur dev.

En production, je surveillerais les performances sur un échantillon récent annoté et j'ajusterais la calibration et le seuil sur une validation représentative. Un réentraînement serait envisagé si la dégradation persistait.

Q11. Multilingue (E2, F2) : les données anglaises aident-elles l'arabe ? Que vaut le modèle arabe appliqué aux tweets anglais, et inversement ?

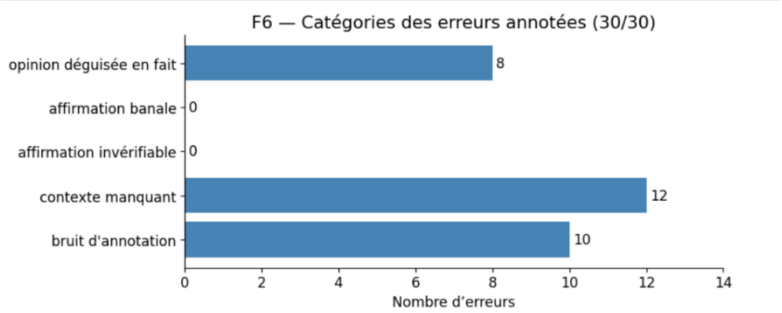
Les checkpoints de la première exécution n'ayant pas été sauvegardés, j'ai réentraîné uniquement la partie E2 pour réaliser les évaluations multilingues de Q11, **en conservant le même protocole, les mêmes hyperparamètres et les mêmes graines**. Les résultats présentés pour cette question proviennent de cette nouvelle exécution.

	modele	tete	langue_test	graines	f1_moyen	f1_ecart_type	precision_moyenne	rappel_moyen	pr_auc_moyen	macro_f1_moyen
0	E2_XLMR_lambda_0.5	ar	ar	3	0.5448	0.0148	0.3962	0.8746	0.4256	0.4653
1	E2_XLMR_lambda_0.5	ar	en	3	0.6867	0.0609	0.7138	0.6818	0.7163	0.7909
2	E2_XLMR_lambda_0.5	en	ar	3	0.4562	0.1536	0.4296	0.5979	0.4448	0.5202
3	E2_XLMR_lambda_0.5	en	en	3	0.6831	0.0426	0.8000	0.6023	0.8155	0.7954

La **tête arabe appliquée aux tweets anglais** obtient un F1 moyen de **0,6867**, proche de celui de la tête anglaise sur ces mêmes données (**0,6831**). Dans l'autre sens, la **tête anglaise appliquée aux tweets arabes** atteint **0,4562**, contre **0,5448** pour la tête arabe, avec une variabilité plus forte. Le transfert fonctionne donc mieux de la tête arabe vers les tweets anglais que dans le sens inverse. Malgré un corpus anglais plus volumineux, davantage de données ne garantit pas un meilleur transfert, notamment lorsque les langues, les thèmes et les formulations diffèrent. Les deux têtes partagent l'encodeur XLM-R, mais la perte **L = L_arabe + 0,5 × L_anglais** donne un poids plus élevé à la tâche

arabe. Ces éléments peuvent contribuer aux différences observées, sans permettre d'en établir précisément les causes.

Q12. Analyse d'erreurs (F6) : catégories dominantes, part du bruit d'annotation. Quel modèle mettriez-vous en production, et pourquoi (qualité, robustesse, coût, latence — chiffres à l'appui) ?



L'analyse manuelle de **30 erreurs d'AraBERT** montre que les catégories dominantes sont le **contexte manquant : 12 cas (40 %)**, le **bruit d'annotation estimé : 10 cas (33,33 %)** et les **opinions déguisées en faits : 8 cas (26,67 %)**. Le contexte joue donc un rôle important dans les erreurs. La part du bruit d'annotation concerne uniquement cet échantillon et repose sur mon jugement manuel ; elle ne peut pas être généralisée à tout le dataset.

	partie	modele	graines	f1_dev_moyen	f1_dev_test_moyen	f1_test_moyen	f1_test_ecart_type	pr_auc_test_moyen	macro_f1_test_moyen	ecart_test_moins_dev_test	temps_entrainement_moyen_s	latence_CPU_ms_exemple	taille_poids_Mio
0	A	TFIDF_LR_seuil_optimise	1	0.5574	0.8644	0.5365	NaN	0.3484	0.3363	-0.3279	17.1947	11.6385	5.9791
1	B1	NLI_fr_optimise	1	0.5469	0.8584	0.5266	NaN	0.3353	0.2633	-0.3319	0.0000	8842.9701	531.7910
2	B2	CAMeLBERT_entite_et_nombre	1	0.3351	0.5261	0.3503	NaN	0.4630	0.4564	-0.1758	0.0000	198.2823	413.8858
3	C	C_meilleur_ponderee_optimise	1	0.5694	0.8647	0.5342	NaN	0.3923	0.3153	-0.3305	9.6328	2.1555	18.8959
4	D	D2_BiLSTM_attention_optimise	1	0.5922	0.8747	0.5231	NaN	0.3781	0.4491	-0.3516	43.5310	174.9235	517.4146
5	E1	E1_AraBERT_optimise	3	0.6038	0.8419	0.5393	0.0081	0.3943	0.4384	-0.3025	293.9673	171.7106	515.7246
6	E2	E2_XLMR_lambda_0.5_optimise	3	0.5957	0.8594	0.5451	0.0149	0.4256	0.4660	-0.3143	805.0873	279.3004	1060.6582

TF-IDF + LR reste proche d'AraBERT et d'E2, tout en étant beaucoup plus léger. Son entraînement prend environ 17,19 s, contre 293,97 s pour AraBERT et 805,09 s pour E2 dans l'environnement utilisé. C3 serait également intéressant si la priorité était la latence.

Je privilégierais **AraBERT**, car il prend en compte le contexte et les relations entre les mots, alors que TF-IDF repose surtout sur des indices lexicaux. Ce choix me paraît donc plus adapté à l'analyse du sens des tweets. Dans mes expériences, AraBERT obtient un meilleur **macro-F1 test : 0,4384 contre 0,3363**, avec un F1-Yes proche de TF-IDF (**0,5393 contre 0,5365**). Cette capacité contextuelle implique toutefois un coût supérieur : **171,71 ms par tweet sur CPU** et **515,72 Mio**. Avant le déploiement